

Introdução a Algoritmos

Aula 7 – Funções

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Definir e chamar funções com parâmetros e valor de retorno, identificando cada parte da assinatura pelo nome correto;
2. Explicar o que é escopo local e por que variáveis de funções diferentes com o mesmo nome não colidem;
3. Demonstrar experimentalmente que passagem por valor não modifica variáveis do chamador, e explicar por que isso ocorre;
4. Organizar um programa em funções com responsabilidade única, reduzindo duplicação de código.

1 A quinta peça do quebra-cabeça

Na Aula 1, apresentamos cinco peças fundamentais da programação: variável, sequência, decisão, repetição e função. Nas aulas seguintes dominamos as quatro primeiras. Esta aula entrega a última.

Até agora, todos os nossos programas vivem inteiramente dentro de `main`. Isso funciona para programas curtos, mas cria um problema sério à medida que os programas crescem: o mesmo trecho de código precisa ser escrito várias vezes, em lugares diferentes. Quando esse trecho contém um bug, é preciso corrigi-lo em todos os lugares — e é fácil esquecer um.

Considere um programa que precisa calcular a média de três turmas diferentes:

```
/* Turma A */
float soma_a = 0;
int i = 0;
while (i < n_a) { soma_a += nota_a[i]; i++; }
float media_a = soma_a / n_a;

/* Turma B – o mesmo código, copiado */
float soma_b = 0;
i = 0;
while (i < n_b) { soma_b += nota_b[i]; i++; }
float media_b = soma_b / n_b;

/* Turma C – copiado de novo */
float soma_c = 0;
i = 0;
```

```
while (i < n_c) { soma_c += nota_c[i]; i++; }  
float media_c = soma_c / n_c;
```

Três cópias do mesmo raciocínio. Se o cálculo estiver errado — digamos, deveria ser média ponderada — precisamos corrigir três lugares. Se esquecermos um, o programa fica inconsistente. Isso é exatamente o tipo de situação que funções foram criadas para eliminar.

Definição — Função

Uma função é um bloco de instruções com nome, que recebe zero ou mais **parâmetros** de entrada, executa um processamento e opcionalmente **retorna** um valor ao código que a chamou. Uma vez definida, pode ser chamada quantas vezes forem necessárias, de qualquer parte do programa.

2 Anatomia de uma função

```
float media(float soma, int n) {  
    return soma / n;  
}
```

Cada parte tem um nome e um papel:

Parte	No exemplo	Significado
Tipo de retorno	float	O tipo do valor que a função devolve ao chamador. Se a função não devolve nada, usa-se void.
Nome	media	Identificador pelo qual a função é chamada. As mesmas regras de nomes de variáveis se aplicam.
Parâmetros	float soma, int n	Variáveis que recebem os valores fornecidos pelo chamador. Cada parâmetro tem tipo e nome, separados por vírgula.
Corpo	{ return soma / n; }	O bloco de instruções que define o que a função faz.
Comando return	return soma / n;	Encerra a função e devolve o valor indicado ao chamador. O tipo do valor deve ser compatível com o tipo de retorno declarado.

Uma função pode ter zero parâmetros e pode não retornar nada:

```
void imprime_separador() {  
    printf("-----\n");  
}
```

E pode ter quantos parâmetros forem necessários:

```
float imc(float peso, float altura) {  
    return peso / (altura * altura);  
}
```

3 Como a chamada de função funciona

Quando o programa chama uma função, a execução **salta** para o início do corpo da função, executa todas as suas instruções e, ao encontrar `return`, **volta exatamente para o ponto da chamada** — trazendo o valor de retorno.

```
#include <stdio.h>

float media(float soma, int n) {          /* (3) executa aqui */
    return soma / n;                     /* (4) calcula e retorna */
}

int main() {
    float s = 45.0;
    int q = 3;
    float m;

    m = media(s, q);                     /* (1) chama media(45.0, 3) */
                                         /* (2) execução salta para media */
                                         /* (5) retorna 15.0; m recebe 15.0 */
    printf("%.1f\n", m);                 /* (6) continua aqui: imprime 15.0 */
    return 0;
}
```

Os comentários numerados marcam a ordem de execução. O fluxo não é linear — ele salta para `media` e volta — mas é completamente determinístico e previsível.

Quando a função é chamada, os valores de `s` e `q` são **copiados** para os parâmetros `soma` e `n`. A função trabalha com essas cópias. Isso é a **passagem por valor**, e suas consequências serão exploradas na Seção 6.

4 Escopo de variáveis

Variáveis declaradas dentro de uma função — incluindo seus parâmetros — existem **apenas enquanto a função está executando**. Elas são criadas quando a função é chamada e destruídas quando ela retorna. Esse princípio chama-se **escopo local**.

```
float media(float soma, int n) {
    float resultado = soma / n; /* resultado existe só aqui dentro */
    return resultado;
}

int main() {
    float m = media(30.0, 3);
    /* resultado não existe aqui — tentar acessá-la seria erro */
    printf("%.1f\n", m);
    return 0;
}
```

Isso significa que duas funções diferentes podem ter variáveis com o mesmo nome sem nenhum conflito — cada uma enxerga apenas as suas próprias:

```
float dobro(float x) {
    float resultado = x * 2; /* este resultado... */
    return resultado;
}
```

```
}

float quadrado(float x) {
    float resultado = x * x; /* ...e este resultado são variáveis distintas */
    return resultado;
}
```

Escopo local é uma das propriedades mais importantes das funções. Ele garante que uma função não interfere acidentalmente nas variáveis de outra — cada função é um ambiente isolado. Isso torna o programa muito mais fácil de raciocinar e depurar.

5 Protótipos de função

Em C, uma função precisa ser **conhecida** pelo compilador antes de ser chamada. Há duas formas de garantir isso.

A primeira é definir a função **antes** de main — que é o que fizemos até agora. A segunda é usar um **protótipo**: uma declaração antecipada que informa ao compilador o nome, os tipos dos parâmetros e o tipo de retorno, sem fornecer o corpo.

```
#include <stdio.h>

/* Protótipos — declaram as funções antes de main */
float media(float soma, int n);
float imc(float peso, float altura);

int main() {
    printf("%.2f\n", media(90.0, 3));
    printf("%.2f\n", imc(70.0, 1.75));
    return 0;
}

/* Definições completas — podem vir depois de main */
float media(float soma, int n) {
    return soma / n;
}

float imc(float peso, float altura) {
    return peso / (altura * altura);
}
```

Protótipos são a prática padrão em programas maiores: eles permitem organizar as funções na ordem que fizer mais sentido para o leitor, independentemente da ordem que o compilador precisa.

6 Funções que chamam funções

Uma função pode chamar outras funções — inclusive a si mesma (recursão, que veremos em aulas futuras). Isso permite construir soluções em camadas, onde cada função resolve um subproblema específico.

Exemplo — IMC com classificação

```
#include <stdio.h>

float calcula_imc(float peso, float altura) {
    return peso / (altura * altura);
}

void classifica_imc(float imc) {
    if (imc < 18.5)
        printf("Abaixo do peso\n");
    else if (imc < 25.0)
        printf("Peso normal\n");
    else if (imc < 30.0)
        printf("Sobrepeso\n");
    else
        printf("Obesidade\n");
}

int main() {
    float peso, altura;
    printf("Peso (kg) e altura (m): ");
    scanf("%f %f", &peso, &altura);

    float imc = calcula_imc(peso, altura);
    printf("IMC: %.1f - ", imc);
    classifica_imc(imc);

    return 0;
}
```

main não sabe **como** o IMC é calculado nem **como** a classificação é feita — ela apenas chama as funções responsáveis por cada tarefa. Essa separação de responsabilidades é o coração da boa programação.

7 Boas práticas ao escrever funções

Saber a sintaxe de funções não é suficiente — é preciso saber **quando** e **como** usá-las bem. Três princípios guiam esse julgamento.

Uma função, uma responsabilidade. Uma função deve fazer exatamente uma coisa e fazê-la bem. `calcula_imc` calcula o IMC — não imprime, não classifica, não lê entrada. Quando uma função faz muitas coisas, ela é difícil de testar, difícil de reutilizar e difícil de entender.

O nome deve descrever o que a função faz, não como. `media_ponderada` é um bom nome; `soma_divide_por_n` descreve a implementação (que pode mudar) em vez do comportamento (que é estável).

Uma função deve caber numa tela. Se você precisa rolar para ler o corpo inteiro de uma função, ela provavelmente está fazendo coisas demais. Quebre-a em funções menores.

Funções são a principal ferramenta para combater a complexidade em programas. Um programa bem estruturado em funções pode ser entendido em dois níveis: o nível de `main` (o que o programa

faz, em termos de chamadas de alto nível) e o nível de cada função (como cada tarefa específica é realizada). O leitor escolhe o nível de detalhe que precisa.

8 O limite da passagem por valor

Vimos que quando uma função é chamada, os valores dos argumentos são **copiados** para os parâmetros. A função trabalha com cópias — nunca com as variáveis originais do chamador.

Isso é útil: garante que uma função não modifica acidentalmente variáveis externas. Mas às vezes queremos exatamente isso — modificar variáveis do chamador. O experimento a seguir revela esse limite com clareza.

Experimento — Tentar trocar dois valores dentro de uma função

O programa abaixo pretende trocar os valores de *x* e *y* em *main* chamando a função *troca*:

```
#include <stdio.h>

void troca(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("Dentro de troca: a=%d, b=%d\n", a, b);
}

int main() {
    int x = 10, y = 20;
    printf("Antes: x=%d, y=%d\n", x, y);
    troca(x, y);
    printf("Depois: x=%d, y=%d\n", x, y);
    return 0;
}
```

Saída:

```
Antes:          x=10, y=20
Dentro de troca: a=20, b=10
Depois:         x=10, y=20
```

A troca acontece **dentro de troca** — mas *x* e *y* em *main* não mudam. Por quê?

A explicação está no modelo de passagem por valor. Quando *main* chama *troca(x, y)*, o que acontece é:

1. O valor de *x* (10) é **copiado** para o parâmetro *a*.
2. O valor de *y* (20) é **copiado** para o parâmetro *b*.
3. A função *troca* *a* e *b* — mas *a* e *b* são cópias locais.
4. Quando *troca* retorna, *a* e *b* são destruídos. *x* e *y* nunca foram tocados.

Podemos visualizar isso como dois “mundos” separados na memória:

Mundo de main $x \rightarrow 10$ (*nunca muda*) $y \rightarrow 20$ (*nunca muda*)**Mundo de troca (cópias)** $a \rightarrow 10 \rightarrow 20$ (*descartado*) $b \rightarrow 20 \rightarrow 10$ (*descartado*)

Para que uma função **modifique** variáveis do chamador, ela precisa de acesso direto ao endereço de memória dessas variáveis — não a cópias dos valores. Esse mecanismo chama-se **passagem por referência** e é implementado em C por meio de **ponteiros**, que estudaremos na próxima aula.

Atenção

O experimento da troca é uma das lições mais importantes desta aula. Guarde bem a conclusão: **em C, uma função nunca modifica as variáveis do chamador por passagem de valor**. Se o seu programa depende de uma função modificar uma variável externa e isso não está funcionando, quase certamente o problema é esse. A solução — ponteiros — vem na Aula 8.

9 Exemplo integrador: busca exaustiva com funções

Na Aula 6, o programa de troco mínimo tinha todo o seu raciocínio dentro de `main`. Vamos reescrevê-lo usando funções para separar responsabilidades. Compare os dois e observe como a versão com funções é mais fácil de ler — cada função tem um nome que descreve o que ela faz, e `main` conta a história do programa em alto nível.

```
#include <stdio.h>

/*
 * Retorna o número de moedas de 10c, 5c e 1c
 * necessárias para compor o valor v usando a
 * combinação (a moedas de 10c, b moedas de 5c).
 */
int total_moedas(int a, int b, int v) {
    int c = v - 10*a - 5*b;
    return a + b + c;
}

/*
 * Retorna 1 se a combinação (a, b) compõe exatamente
 * o valor v com moedas não-negativas; 0 caso contrário.
 */
int combinacao_valida(int a, int b, int v) {
    int c = v - 10*a - 5*b;
    return c >= 0;
}

/*
 * Encontra e imprime a combinação de moedas de 10c, 5c
 * e 1c que compõe v centavos com o menor número de moedas.
 */
void troco_minimo(int v) {
    int melhor = v + 1;
    int a_melhor = 0, b_melhor = 0;
```

```
for (int a = 0; a <= v / 10; a++) {
    for (int b = 0; b <= (v - 10*a) / 5; b++) {
        if (combinacao_valida(a, b, v)) {
            int total = total_moedas(a, b, v);
            if (total < melhor) {
                melhor = total;
                a_melhor = a;
                b_melhor = b;
            }
        }
    }
}

int c_melhor = v - 10*a_melhor - 5*b_melhor;
printf("Mínimo: %d moeda(s)\n", melhor);
printf(" 10c: %d\n   5c: %d\n   1c: %d\n",
        a_melhor, b_melhor, c_melhor);
}

int main() {
    int v;
    printf("Troco (centavos): ");
    scanf("%d", &v);
    troco_minimo(v);
    return 0;
}
```

Observe que `main` agora tem apenas três linhas de lógica. Ela não sabe como o troco é calculado — apenas chama `troco_minimo` e confia que ela faz o trabalho. Isso é **abstração**: esconder os detalhes de implementação atrás de um nome descritivo.

10 Conclusão

Funções são a última peça do quebra-cabeça apresentado na Aula 1 — e a mais transformadora. Com elas, um programa deixa de ser uma lista longa de instruções e passa a ser uma coleção de **subproblemas resolvidos**, cada um com nome e responsabilidade clara.

Os conceitos centrais desta aula:

1. **Definição e chamada:** uma função é definida uma vez e chamada quantas vezes forem necessárias, de qualquer lugar do programa.
2. **Escopo local:** variáveis de uma função existem apenas enquanto ela executa. Funções são ambientes isolados — não interferem umas nas outras.
3. **Passagem por valor:** argumentos são copiados para os parâmetros. A função trabalha com cópias e nunca modifica as variáveis originais do chamador.
4. **O limite da passagem por valor:** funções não conseguem modificar variáveis externas por esse mecanismo. Para isso, precisamos de ponteiros — tema da próxima aula.

11 Exercícios

1. Escreva uma função `int eh_primo(int n)` que retorna 1 se `n` é primo e 0 caso contrário. Use-a num programa que leia um inteiro e imprima “primo” ou “não primo”. (*Reaproveite o raciocínio do exercício de desafio da Aula 5.*)
2. Escreva uma função `int maior(int a, int b)` que retorna o maior entre dois inteiros. Em seguida, escreva `int maior3(int a, int b, int c)` que retorna o maior entre três inteiros **chamando maior duas vezes** — sem usar `if` diretamente em `maior3`.
3. Escreva funções `float celsius_para_fahrenheit(float c)` e `float fahrenheit_para_celsius(float f)`. Escreva um programa que leia uma temperatura e uma letra ('C' ou 'F') e converta para a outra escala.
4. O programa abaixo pretende calcular o dobro de `x` modificando-o dentro de uma função. Explique por que não funciona e como o comportamento pode ser obtido corretamente **sem ponteiros** (dica: use o valor de retorno).

```
void dobra(int x) {  
    x = x * 2;  
}  
int main() {  
    int v = 5;  
    dobra(v);  
    printf("%d\n", v);    /* imprime 5, não 10 */  
    return 0;  
}
```

5. (*Desafio*) Escreva uma função `int mdc(int a, int b)` que calcula o máximo divisor comum de dois inteiros usando o algoritmo de Euclides: enquanto `b != 0`, substitua `(a, b)` por `(b, a % b)`; quando `b == 0`, `a` é o MDC. Use-a para calcular o MMC de dois números (`mmc = a * b / mdc(a, b)`).